Heart Disease Risk Prediction: An End-to-End MLOps Pipeline

Course: Machine Learning Operations (AIMLCZG523), Assignment 1 | **Student:** Rahul Yadav | **BITS ID:** 2025ae05947 (2025ae05947@wilp.bits-pilani.ac.in)

Code repository: https://github.com//heart-disease-mlops

1. Introduction and problem statement

The goal is a classifier that estimates the risk of heart disease from routine patient measurements, wrapped in the automation a production service needs: reproducible training, experiment tracking, tests, CI, a container image, a Kubernetes deployment and basic monitoring. The model is deliberately kept simple; most of the effort goes into making the path from raw data to a served prediction repeatable and observable.

The data is the UCI Heart Disease dataset. The Cleveland subset (303 patients, 13 predictors plus a 0-4 severity label) is used because it is the standard, most complete subset. The label is binarised: 0 means no disease and 1-4 means disease present.

2. Setup and installation

```
git clone <repo-url> && cd heart-disease-mlops
python -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt

python scripts/download_data.py        # fetch UCI data (or --zip heartdisease.zip)
python -m src.heart.eda                # cleaned CSV + figures in reports/figures
python -m src.heart.train              # tune, evaluate, track, save models/heart_pipeline.joblib
pytest -v                              # unit tests
python -m src.api.app                  # API on :8000  (or: docker build -t heart-api . )
mlflow ui --backend-store-uri ./mlruns # browse experiments (when MLflow is installed)
```

All scripts run from a clean virtual environment using `requirements.txt`. The Docker image uses the slimmer `requirements-serve.txt`, which leaves out plotting and MLflow.

3. Data acquisition, cleaning and EDA

`scripts/download_data.py` downloads the official zip from the UCI repository (or unpacks a local copy with `--zip`) and extracts `processed.cleveland.data`. `src/heart/data.py` then cleans it: `?` is read as missing, the 0-4 label becomes a binary `target`, duplicates are dropped and all columns are typed numerically. The cleaned file is written to `data/processed/heart_clean.csv`.

Only 6 values are missing (4 in `ca`, 2 in `thal`). They are deliberately **not** imputed during cleaning. Imputation (median for numeric, most frequent for categorical) is a step inside the model pipeline, so it is fitted on training folds only and no information leaks from validation data. The API applies the same imputation at serving time.

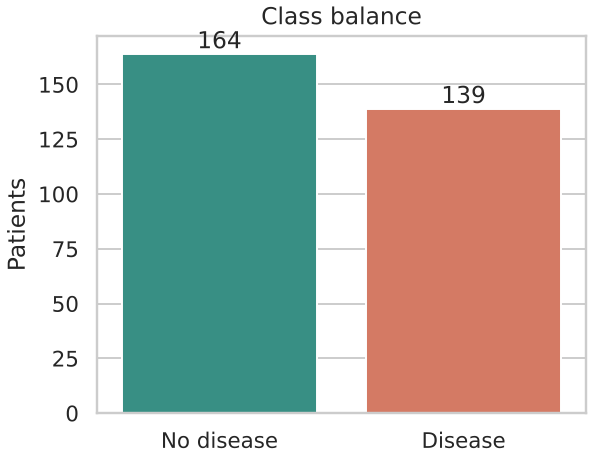


Figure 1. Class balance: 164 healthy vs 139 with disease (54% / 46%). The classes are close to balanced, so accuracy is meaningful and no resampling is needed.

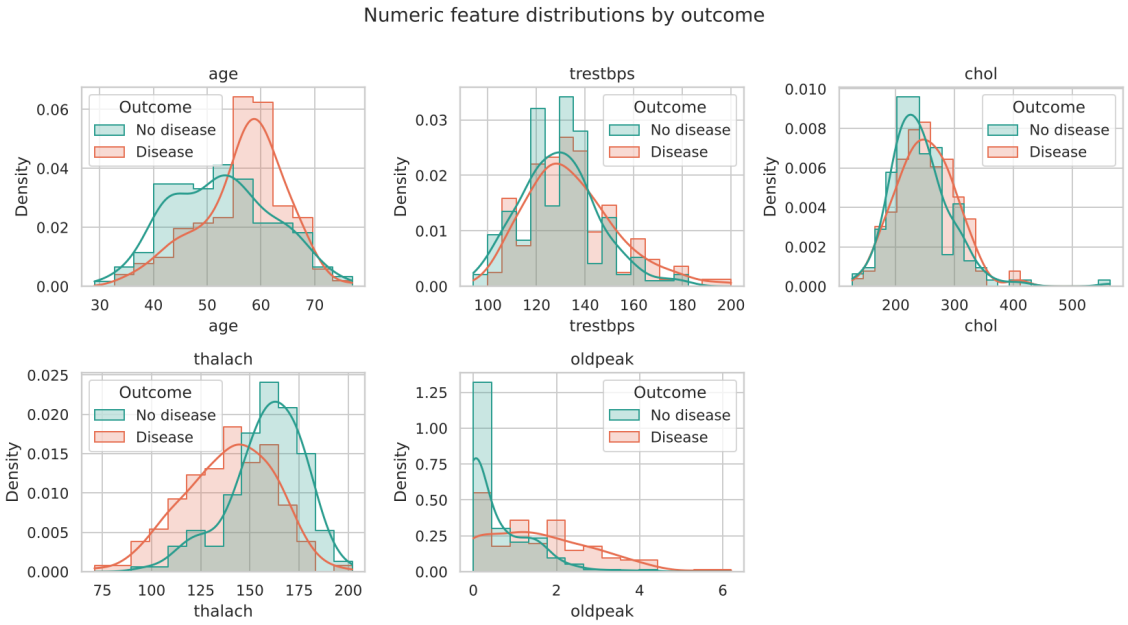


Figure 2. Numeric features by outcome. Patients with disease reach lower peak heart rates (thalach) and show higher ST depression (oldpeak).

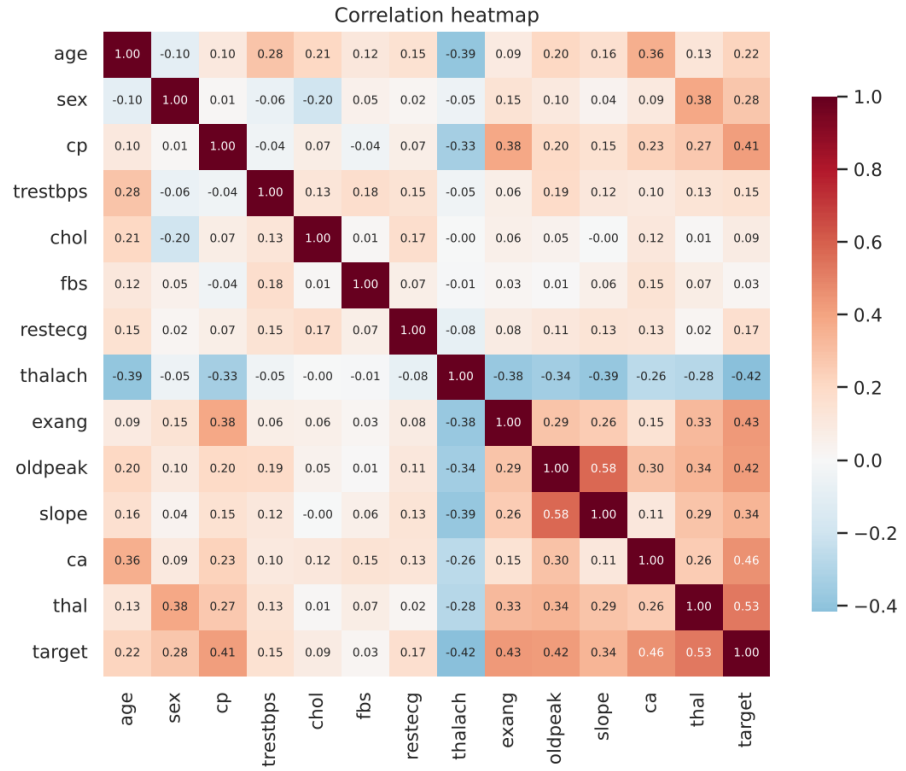


Figure 3. Correlation heatmap. The strongest links to the target are thal (0.53), ca (0.46), exang (0.43), oldpeak (0.42), thalach (-0.42) and cp (0.41). oldpeak and slope are correlated with each other (0.58), but not enough to need removing.

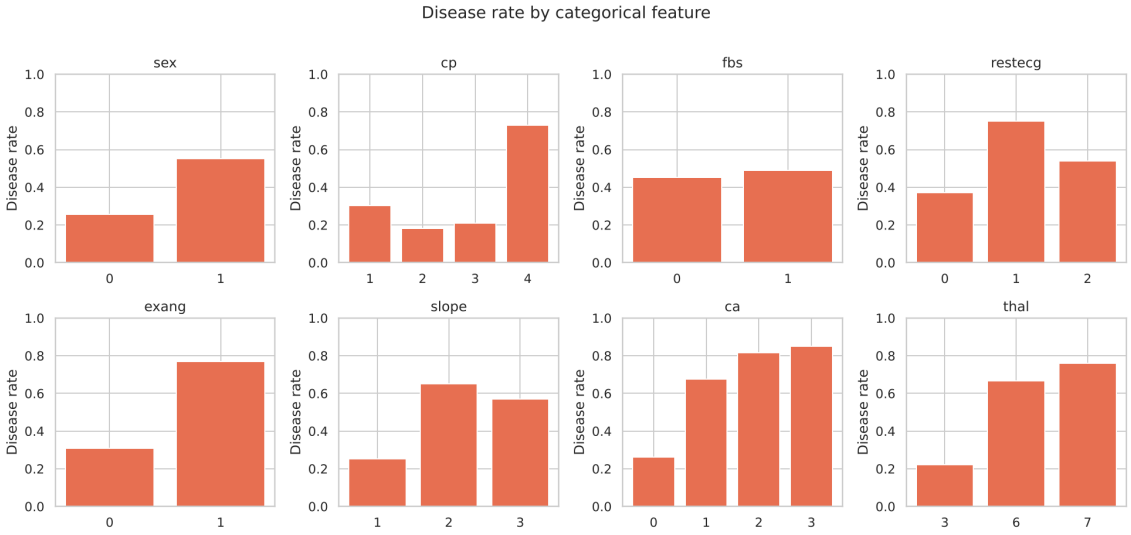


Figure 4. Disease rate per category. Asymptomatic chest pain (cp=4), exercise-induced angina (exang=1), reversible-defect thalassemia (thal=7) and a higher vessel count (ca) all carry a much higher disease rate.

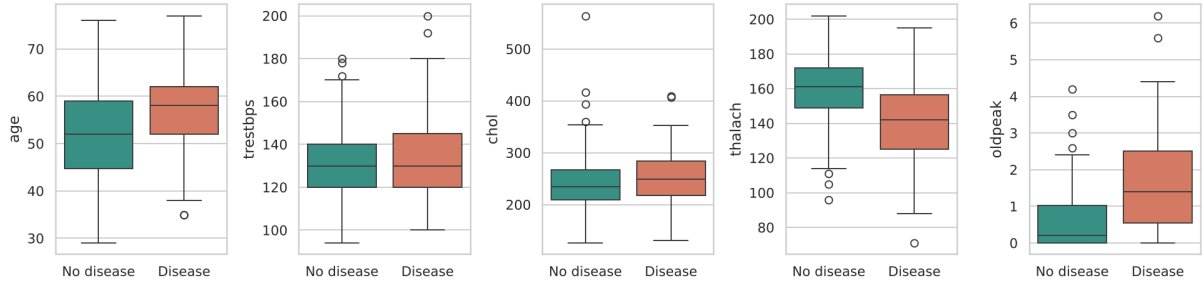


Figure 4b. Boxplots of numeric features by outcome (outlier inspection).

The boxplots show a few high-cholesterol and high-oldpeak outliers. They are plausible clinical values rather than errors, so they are kept. Scaling plus a regularised linear model keeps their influence bounded.

4. Feature engineering and model development

4.1 Preprocessing pipeline

- **Numeric** (age, trestbps, chol, thalach, oldpeak + engineered): median imputation, then standardisation.
- **Categorical** (sex, cp, fbs, restecg, exang, slope, ca, thal): most-frequent imputation, then one-hot encoding with `handle_unknown='ignore'`, so an unseen category at serving time cannot crash the API.
- **Engineered feature:** `hr_reserve_ratio = thalach / (220 - age)`, the peak heart rate achieved as a fraction of the age-predicted maximum. It captures the clinical idea that a heart rate which is low *for the patient's age* is a warning sign, something neither column shows alone.

The three stages (feature engineering, preprocessing, model) are one scikit-learn `Pipeline`, so training, cross-validation and serving all share a single code path.

4.2 Models, tuning and selection

Three classifiers were compared: Logistic Regression, Random Forest and Gradient Boosting. The data was split 80/20 (stratified, seed 42), giving 242 training and 61 held-out test patients. Each model was tuned with a grid search on the training split using stratified 5-fold cross-validation, scored by ROC-AUC. The best configuration was then re-evaluated with 5-fold cross-validation on accuracy, precision, recall, F1 and ROC-AUC, and finally scored once on the untouched test set.

Model	Best hyper-parameters (grid search, 5-fold CV, ROC-AUC)
logistic_regression	C=1, class_weight=None
random_forest	max_depth=3, min_samples_leaf=3, n_estimators=400
gradient_boosting	learning_rate=0.05, max_depth=3, n_estimators=50

Model	CV acc	CV prec	CV recall	CV ROC-AUC	Test acc	Test recall	Test ROC-AUC
logistic_regression	0.847	0.877	0.783	0.902	0.885	0.929	0.966
random_forest	0.818	0.843	0.755	0.902	0.836	0.857	0.957
gradient_boosting	0.806	0.828	0.738	0.882	0.902	0.929	0.943

Selection. Models are chosen on cross-validated ROC-AUC from the training split, never on the test set. Logistic Regression and Random Forest are practically tied (0.902) and Logistic Regression is marginally ahead on the unrounded value. It is also the simpler, faster and more interpretable model, so **logistic_regression** is the production model. Its test ROC-AUC is 0.966 with recall 0.929. For a screening tool, recall matters most because a missed case costs more than a false alarm.

Caveat. The test set holds only 61 patients, so one patient moves accuracy by about 1.6 points. That is why Gradient Boosting can show the highest test accuracy (0.902) while having the lowest cross-validated score. The cross-validated numbers are the more trustworthy ones. The model is a decision-support prototype, not a clinical device.

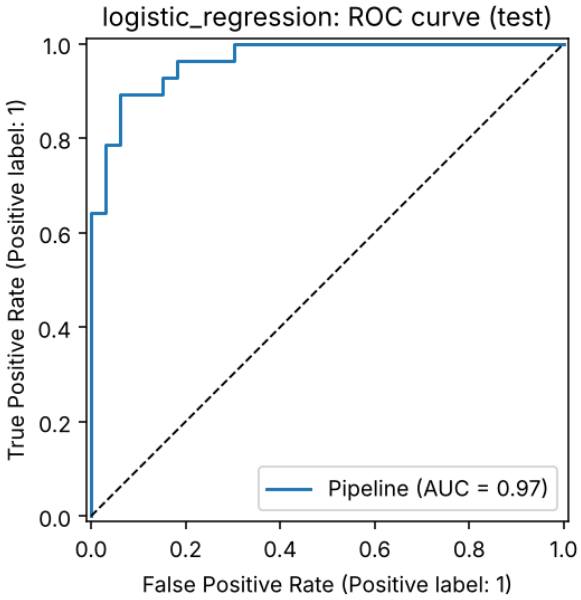


Figure 5. ROC curve of the selected model on the test split.

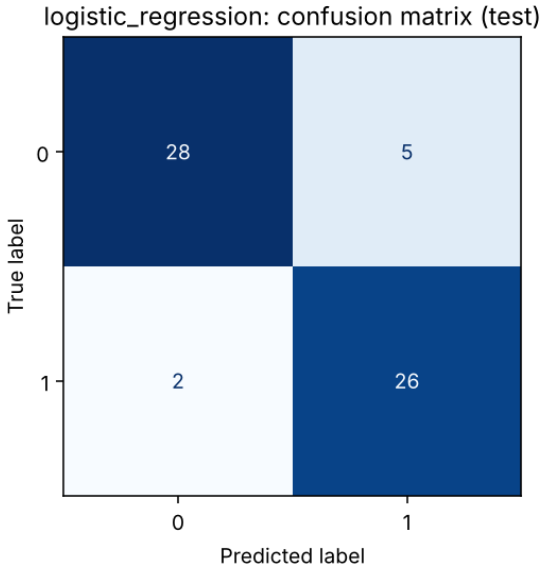


Figure 6. Confusion matrix of the selected model on the test split.

5. Experiment tracking

`src/heart/tracking.py` wraps MLflow. Every candidate model gets its own run in the experiment `heart-disease-risk`, which logs:

- **Parameters:** model name, CV folds, split size, seed, training size and the best hyper-parameters.
- **Metrics:** best grid-search score, CV mean and standard deviation for all five metrics, and test metrics.
- **Artifacts:** ROC curve, confusion matrix and the fitted pipeline. A final `final_selected_model` run also stores the model metadata JSON.

If MLflow is not installed, the wrapper falls back to JSON run files under `mlruns_local/` with the same content, so training never fails because of tracking. **Note:** the results in this report were produced with that fallback, because the build sandbox could not install MLflow. Running `python -m src.heart.train` in a normal environment with `requirements.txt` installed logs to MLflow, and `mlflow ui` then shows the runs. The screenshots of the MLflow UI must be taken on that run.

6. Model packaging and reproducibility

- The full pipeline (imputers, scaler, encoder, model) is saved as one object, `models/heart_pipeline.joblib`, so serving applies exactly the same transformations as training.
- `models/model_metadata.json` records the features, metrics, library versions and the tracking backend.
- `requirements.txt` pins the verified library versions. Seeds are fixed (42) for the split, CV and the models.
- A unit test loads the saved model and checks ROC-AUC > 0.85, which catches a broken or stale artifact.

7. Testing and CI/CD

There are 20 unit tests (all passing, `screenshots/evidence/unit_tests_output.txt`):

- `test_data.py`: target binarisation, missing values kept for the pipeline, de-duplication, schema check, raw-file shape.
- `test_model.py`: the engineered feature is correct, the preprocessor leaves no NaNs and scales to mean 0, an unseen category does not crash, probabilities sum to 1, and the saved model scores above threshold.
- `test_api.py`: input validation (missing, out-of-range, bad type), `/health`, `/predict` (200 and 400) and the `/metrics` counters.

`.github/workflows/ci.yml` runs two jobs. **lint-test-train** runs flake8, pytest (JUnit XML), data download, EDA, training and an API smoke test, then uploads `reports/`, `models/`, `mlruns/` and `logs/` as workflow artifacts. **docker** builds the image, starts the container and calls `/health` and `/predict`. A lint, test or smoke-test failure stops the workflow with the failing step's log, so a broken change cannot reach the image build.

8. Containerisation and API

`Dockerfile` uses `python:3.13-slim`, installs `requirements-serve.txt`, copies only `src/` and `models/`, runs as a non-root user and serves the Flask app with gunicorn on port 8000. A `HEALTHCHECK` polls `/health`.

```
docker build -t heart-api .
docker run -p 8000:8000 heart-api
curl -X POST localhost:8000/predict -H 'Content-Type: application/json' -d @scripts/sample_input.json
-> {"confidence":0.5066,"label":"heart disease likely","prediction":1,"probability_disease":0.5066}
```

Endpoints: `POST /predict` (JSON with the 13 features, returns the prediction, the probability of disease and the confidence), `GET /health` and `GET /metrics`. Inputs are validated against plausible clinical ranges and allowed category codes. Bad requests get HTTP 400 with the list of problems, and a prediction is never made on bad data. `confidence` is the probability of the predicted class. The curl session above was run against the live Flask app (`screenshots/evidence/api_curl_output.txt`).

Field	Meaning	Allowed values
age, trestbps, chol, thalach, oldpeak	age, resting BP, cholesterol, max heart rate, ST depression	age 1-120, trestbps 50-300, chol 50-700, thalach 40-260, oldpeak -3 to 10
sex, fbs, exang	sex, fasting sugar >120, exercise angina	0 or 1
cp, restecg, slope	chest pain type, resting ECG, ST slope	cp 1-4, restecg 0-2, slope 1-3
ca, thal	major vessels coloured, thalassemia	ca 0-3, thal 3, 6 or 7

9. Production deployment

`k8s/deployment.yaml` defines 2 replicas with readiness and liveness probes on `/health`, resource requests and limits, and Prometheus scrape annotations. `k8s/service.yaml` exposes it through a `LoadBalancer` (`k8s/ingress.yaml` is an Ingress alternative). On Minikube or Docker Desktop:

```
minikube start
eval $(minikube docker-env) && docker build -t heart-api:latest .
kubectl apply -f k8s/
minikube tunnel                      # gives the LoadBalancer an external IP
kubectl get pods,svc
curl -X POST http://<EXTERNAL-IP>/predict -H 'Content-Type: application/json' -d @scripts/sample_input.json
```

For a cloud cluster (GKE, EKS, AKS), push the image to a registry, change the `image:` field and apply the same manifests.

10. Monitoring and logging

- **Request logging:** every request is logged as one JSON line (timestamp, request ID, method, path, status, latency). Each prediction logs its input and result. Logs go to stdout (visible with `kubectl logs`) and to a rotating file, `logs/api.log`. The request ID is also returned in the `X-Request-ID` header.
- **Metrics:** `/metrics` exposes Prometheus-format counters (requests by method/path/status, predictions by class, errors) and a `/predict` latency histogram. It is implemented in about 30 lines without extra dependencies.
- **Dashboard:** `docker-compose.yml` starts the API with Prometheus (`monitoring/prometheus.yml`) and Grafana. Useful panels are request rate, error rate, p95 latency (`histogram_quantile(0.95, rate(heart_api_predict_latency_seconds_bucket[5m]))`) and the share of positive predictions. A sudden shift in that share is a cheap early signal of data drift.

11. Architecture

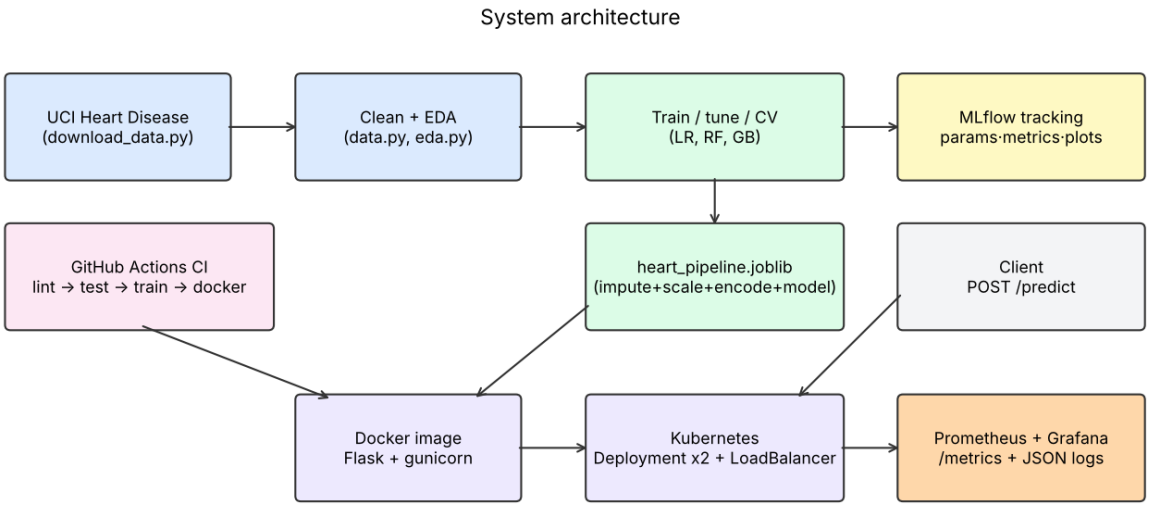


Figure 7. End-to-end architecture: data, training and tracking, CI, container, Kubernetes, monitoring.

12. Verification status and limitations

This table states which parts were executed in the build environment and which need to be run on your machine.

Component	Status
Data download script, cleaning, EDA figures	Executed (data from the supplied zip)
Training, CV, model selection, saved pipeline	Executed; results above
MLflow logging	Code written; run executed with the JSON fallback. Re-run with MLflow installed
Unit tests (20)	Executed, all pass (run with unittest; pytest also discovers them)
API (/predict, /health, /metrics)	Executed against the live Flask app with curl
Dockerfile, docker-compose	Written; image build not run (no Docker daemon available)
GitHub Actions workflow	Written; YAML validated; not run (needs a pushed repo)
Kubernetes manifests, Prometheus, Grafana	Written; YAML validated; not deployed
flake8	Not run (not installable here); manual checks for unused imports and line length

Other limitations. The dataset is small (303 patients, one hospital), so performance may not generalise. No fairness analysis was done across sex or age groups. Predictions are probabilities, not diagnoses.

13. Screenshots to add

Add these to `screenshots/` and reference them here once taken: (1) MLflow UI experiment list and one run page; (2) the green GitHub Actions run with its artifacts; (3) `docker build` and `docker run` plus a curl call; (4) `kubectl get pods,svc` and a curl against the external IP; (5) the Prometheus targets page and the Grafana panel. The EDA, ROC and confusion-matrix figures, unit-test output and curl output are already in the repo.